

“Now that we have all this useful information, it would be nice to be able to do something with it. (Actually, it can be emotionally fulfilling just to get information. This is usually only true, however, if you have the social life of a kumquat.)”

- UNIX Programmer's Manual

CSE 312

Operating Systems

Spring 2026

Memory Management

© 2013-2026 Yakup Genç

1

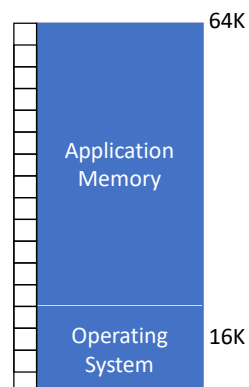
Today

- Memory management...

2

Memory Management

- Main memory is a resource
- Operating systems have to manage it
- Could have been simple: put the OS at the bottom, and the application gets the rest
- Not so...



Mar 20, 2015

CSE312 Lecture 6

3

3

Memory as a Resource

- Memory has always been scarce
- More precisely, memory has generally been expensive
- We've never had enough to satisfy all applications
- We've always had to cope

Mar 20, 2015

CSE312 Lecture 6

4

4

Static Overlays

- Early days: application programmers had to manage memory allocation themselves
- Different subroutines occupied the same area of memory at different times
- The OS provided library routines to support this, but the layout — and the safety of the scheme — was up to the programmer

Mar 20, 2015

CSE312 Lecture 6

5

5

Memory and Multiprogramming

- Memory sizes increased
- We used the increase to support multiprogramming
- Applications still had to manage their own memory
- But the OS had new problems – challenges
 - Relocation
 - Memory protection
 - Allocation

Mar 20, 2015

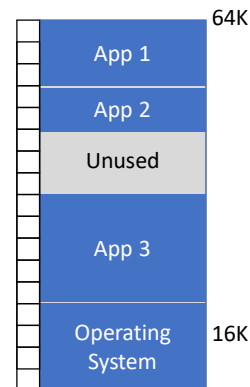
CSE312 Lecture 6

6

6

Relocation

- Originally, all applications started at the same address, perhaps 16K
- With multiprogramming, applications start at different spots
- In fact, a given application could start at a different location each time it ran
- What to do ...
 - Position-independent code
 - Load-time relocation
 - Hardware assist



Mar 20, 2015

CSE312 Lecture 6

7

7

Position-Independent Code (PIC)

- Some code is naturally position-independent
- There are no references to absolute memory locations; everything is relative to a register or the program counter:

```
L    R3, 16(R12)
```

```
B    *-4
```

- Easier on some hardware than others
- Used for shared libraries on most modern Unix variants
- Generally a compiler option today: `-fpic` or `-fPIC` when using `gcc`

Mar 20, 2015

CSE312 Lecture 6

8

8

Load-Time Relocation

- At program load time, the execution address is known
- Add the actual offset to the presumed location specified by the compiler and linker
- The cost of this — that is, the number of words that need to be relocated — varies depending on the hardware architecture and compiler

Mar 20, 2015

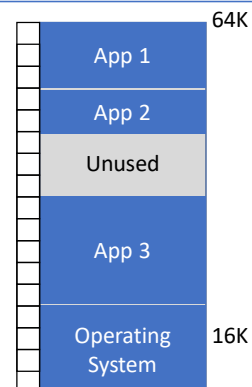
CSE312 Lecture 6

9

9

Hardware-Assisted Relocation

- PIC is sometimes awkward (and doesn't apply after loading)
- Software relocation can be expensive
- We sometimes want to move programs around
- We have 20K free here, but the largest program we can run is 12K



Mar 20, 2015

CSE312 Lecture 6

10

10

Base/Limit Registers

- The base and limit registers are set by the OS
- The contents of the base register is added to all memory references
- Any memory reference larger than the limit register is invalid and causes a trap
- Problems with Base/Limit Registers
 - Copying programs is expensive
 - An extra addition on every memory reference is expensive, too
 - We need more flexibility

Mar 20, 2015

CSE312 Lecture 6

11

11

Swapping

- Write the entire program out to disk when it's blocked
- Read it all back in again, possibly at a different address, when it's ready to run
- Can use this to compact memory
- Apparently frees up the CPU, but disk I/O consumes memory bandwidth

Mar 20, 2015

CSE312 Lecture 6

12

12

Memory Protection

- Memory Protection
- Need some way to keep applications from touching other applications' memory (or the operating system's)
- Base/limit registers handle that automatically
- Another solution: attach protection information to each memory block
- Example: on IBM System/360, each 4K of memory had a 4-bit storage key; it had to match the key in the program status word

Mar 20, 2015

CSE312 Lecture 6

13

13

The Memory Allocation Problem

- The OS knows that certain areas of memory are free
- An application requests some
- What block of memory is given to that application?
- For now, assume a linked list of free blocks

Mar 20, 2015

CSE312 Lecture 6

14

14

First Fit

- Traverse the list until a large-enough block is found
- Allocate as much of it as is needed
- Return the rest to the free list
- When freeing memory, must coalesce adjacent blocks
- Obvious answer: order linked list by address

Mar 20, 2015

CSE312 Lecture 6

15

15

Best Fit

- First fit seems wasteful — there may be an exact match further down the list
- Solution: best fit
- Find the free block that's closest in size to the request
- Obvious data structure: order linked list by size
- Makes coalescence less efficient
- And best fit isn't more efficient!

Mar 20, 2015

CSE312 Lecture 6

16

16

Worst Fit

- Best fit tends to leave small, useless blocks (fragmentation)
- What about worst fit — pick the largest block and sub-divide it?
- That's not very good, it turns out; first fit and best fit are better and are pretty close
- Note: this is a simulation result based on trace data
- For first fit, as much as 1/3 of memory may be unusable!
- Many other algorithms...

Mar 20, 2015

CSE312 Lecture 6

17

17

The Status of Memory Allocation

- Classic memory allocation isn't very important for the OS, because of virtual memory
- But applications still need memory allocation within their OS allocation:
- malloc() in C; new in Java and C++
- The same sorts of algorithms are used, with one important addition: applications can ask the OS for more memory

Mar 20, 2015

CSE312 Lecture 6

18

18

Virtual Memory

- Problems with Earlier Schemes
 - Programmer time wasted on memory allocation
 - Inefficiency of block allocation algorithms
 - Expense of compaction
 - Expense of base registers
 - The need for special memory protection mechanisms
 - The solution: virtual memory

Mar 20, 2015

CSE312 Lecture 6

19

19

Virtual Memory

- Convert a virtual address — the address as seen by the program — to a physical address
- Mapping is done by pages — groups of (perhaps) 4K bytes
- The mapping need not refer to contiguous areas of physical memory
- Eliminates all problems of allocation inefficiency, compaction, relocation, memory protection, and more

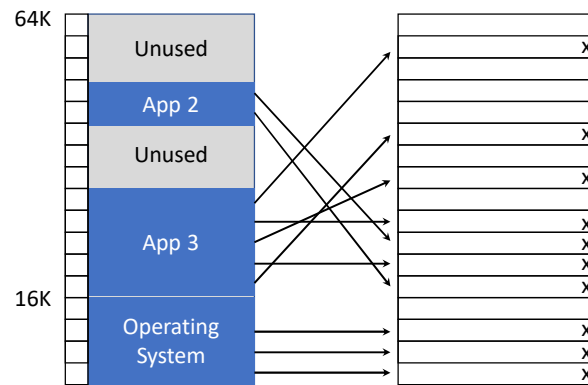
Mar 20, 2015

CSE312 Lecture 6

20

20

Virtual Memory



Mar 20, 2015

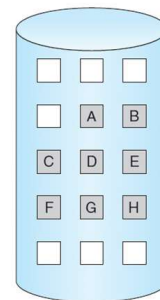
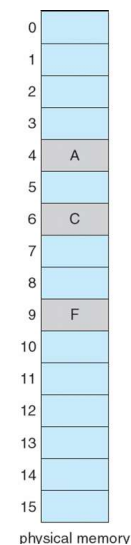
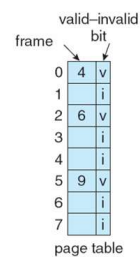
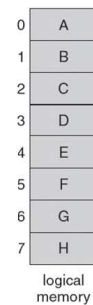
CSE312 Lecture 6

21

21

Virtual Memory Management

- Page table has one entry per page of the process image
- All pages represented on disk
- Only some of them loaded into physical memory



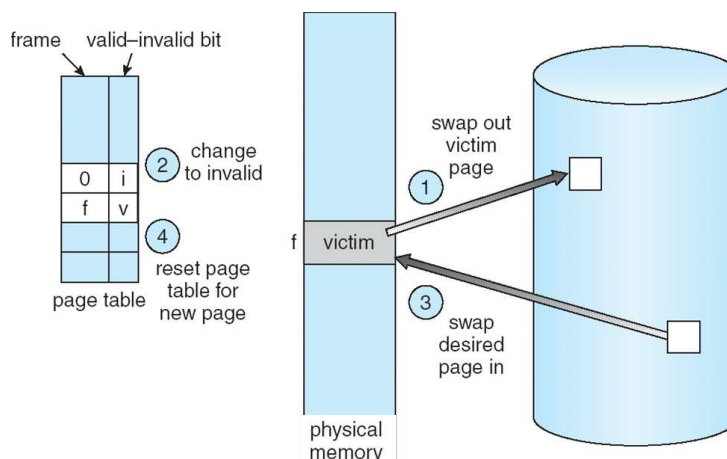
Mar 20, 2015

CSE312 Lecture 6

22

22

Page Replacement



Mar 20, 2015

CSE312 Lecture 6

23

23

Virtual Memory

- Each page occupies a page frame in physical memory
- There is no relationship between a page's virtual address and its physical address
- Each application could, in fact, start at the same virtual address as every other application
- The OS may or may not be visible in the application's address space
- Physical memory that isn't pointed to can't be accessed — no need for special memory protection (but some forms are still used)

Mar 20, 2015

CSE312 Lecture 6

24

24

Implementing Virtual Memory

- Divide a virtual address A into $\langle V, O \rangle$, where V is the virtual page number and O is the offset within the page
- The memory management unit (MMU) maps V into P , the physical page number and produces $\langle P, O \rangle$
- In its simplest form, this mapping is done by direct indexing into an MMU register bank M :

$$A \rightarrow \langle M[V], O \rangle$$

- This scheme can be faster than a base register; no additions are involved

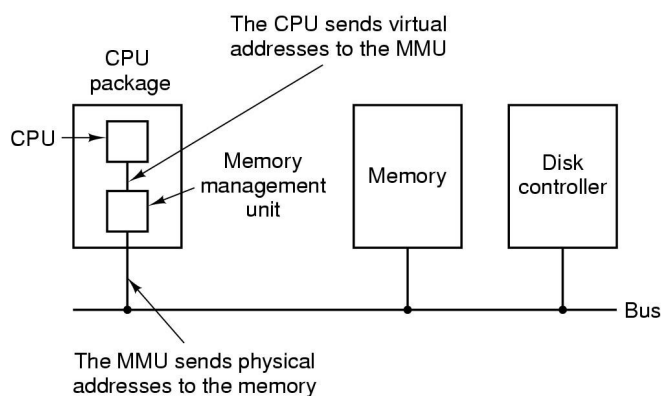
Mar 20, 2015

CSE312 Lecture 6

25

25

Virtual Memory – Paging



The position and function of the MMU – shown as being a part of the CPU chip (it commonly is nowadays). Logically it could be a separate chip, was in years gone by...

Mar 20, 2015

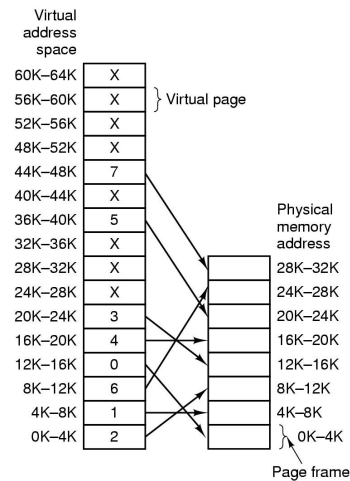
CSE312 Lecture 6

26

26

Virtual Memory – Paging

Relation between virtual addresses and physical memory addresses given by page table



Mar 20, 2015

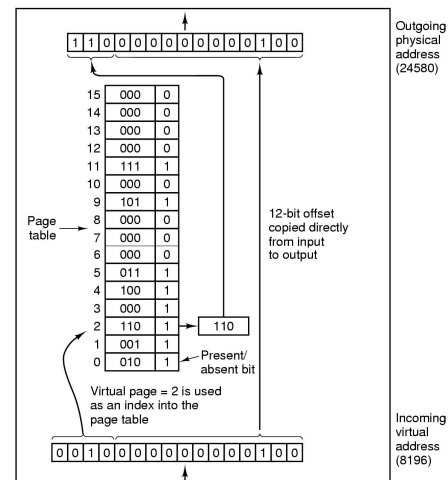
CSE312 Lecture 6

27

27

Virtual Memory – Paging

The internal operation of the MMU with 16 4-KB pages



Mar 20, 2015

CSE312 Lecture 6

28

28

Too Simple

- Earlier times this could work – page sizes were large relative to memory
- The PDP-11s on which Unix grew up had only 8 pages of address space — that many registers were affordable
- Modern computers easily have 1G of memory and pages of 4K bytes – or 256K possible pages
- We can't have that many MMU registers
- We need a separate page table for each process — we can't afford to copy that many MMU registers during process switches

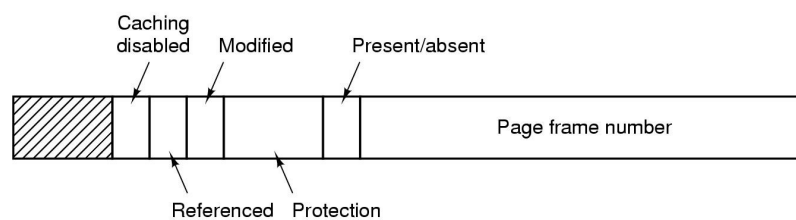
Mar 20, 2015

CSE312 Lecture 6

29

29

Structure of Page Table Entry



A typical page table entry

Mar 20, 2015

CSE312 Lecture 6

30

30

Page Tables in RAM

- First-order solution: put the page table in RAM
- Still problematic:
 - It's slow — an extra RAM access for each memory reference
 - It's a lot of RAM for page tables
- Page tables do live in RAM, but there are optimizations

Mar 20, 2015

CSE312 Lecture 6

31

31

Translation Lookaside Buffer

- Mappings from $V \rightarrow P$ are cached in the *Translation Lookaside Buffer* (TLB)
- The TLB is an associative memory — it maps a value, rather than an index, into another value
- When there's a TLB miss, an old entry is discarded, a memory lookup is done, and the new entry is inserted into the TLB
- This works because programs tend to exhibit locality of reference
- On many systems, the TLB is managed in hardware; on RISC systems, it is explicitly populated by the OS

Mar 20, 2015

CSE312 Lecture 6

32

32

Structure of Page Table Entry

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

A TLB to speed up paging

Mar 20, 2015

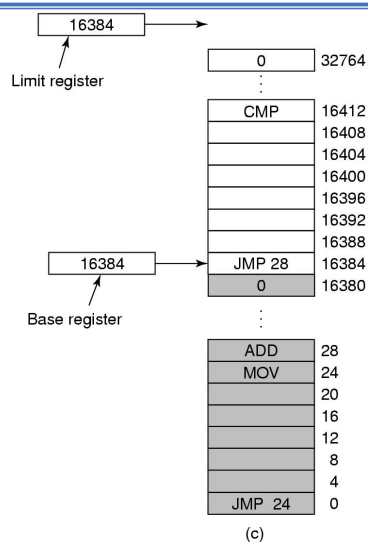
CSE312 Lecture 6

33

33

The Need for Segmentation Tables

- We still don't want to have 256K page table entries
- We could have a limit register — but sometimes, virtual memory is sparse
- Example: put the OS at (virtual) address 2G, the executable at (virtual) address 128M, and have the stack grown down from 192M. On Solaris, applications start (near) 0, but the stack grows down from 4G.
- We need two-level page tables



Mar 20, 2015

CSE312 Lecture 6

34

34

Segmentation Tables

- Multilevel page tables...
- A is treated as $\langle S, V, O \rangle$
- A segment number S points to a page table; V , the page number, is an index into the page table array pointed to by S
- The full translation is thus

$$V \rightarrow \langle R[S[P]], O \rangle$$
 where R is the segment table address
- The TLB is used to speed this up, too

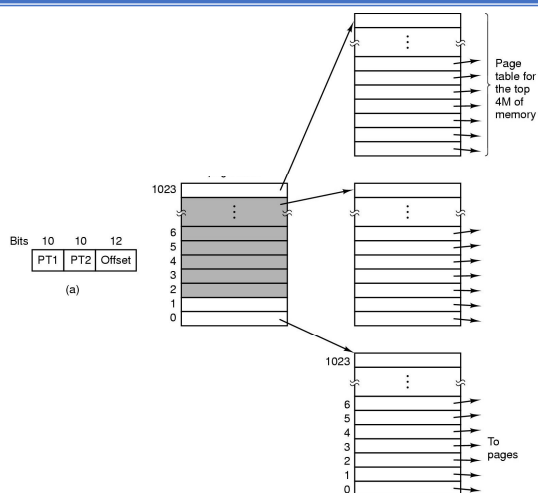
Mar 20, 2015

CSE312 Lecture 6

35

35

Segmentation Tables



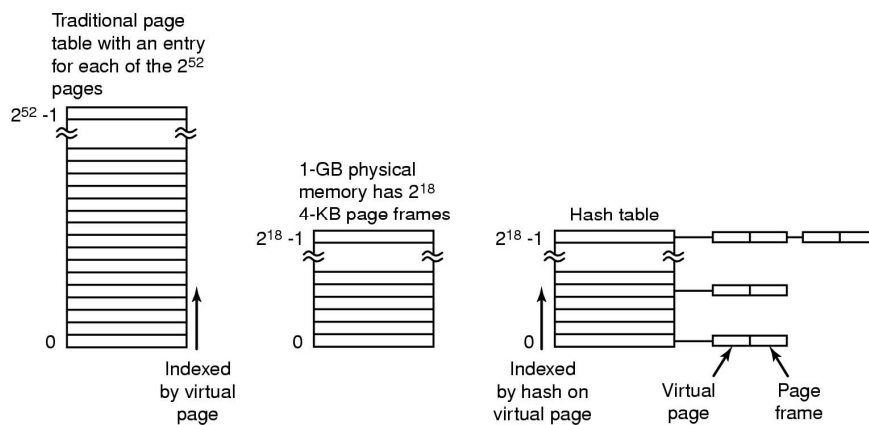
Mar 20, 2015

CSE312 Lecture 6

36

36

Inverted Page Tables



Comparison of a traditional page table with an inverted page table

Mar 20, 2015

CSE312 Lecture 6

37

37

VM and the Cache

- Some caches work on physical memory addresses; others work on virtual addresses
- Either way, the cache is largely useless after a VM map change
- Either the cache has to be invalidated or different physical addresses are used

Mar 20, 2015

CSE312 Lecture 6

38

38

Switching Processes

- Load a single register pointing to the new segment table
- Purge the old TLB (often done explicitly)
- The first reference to memory uses the segment register as find a page table pointer in memory; it then makes a second memory reference to find the physical page number
- Again, locality of reference helps
- For kernel mode, use a full segment table; for user mode, use a segment table that doesn't include the kernel pages
- Memory map changes are extremely expensive because of the effect on the cache and the TLB

Mar 20, 2015

CSE312 Lecture 6

39

39

Segment Table Entries

- A segment table entry has control information as well as a page table address
- In particular, it has a valid/invalid bit
- Unused parts of the address space have invalid segment table entries, and hence no page table

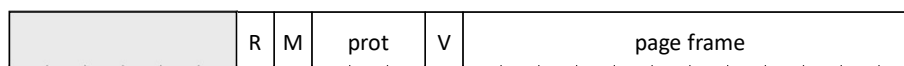
Mar 20, 2015

CSE312 Lecture 6

40

40

Page Table Entries



R page referenced
 M page modified
 prot memory protection: read, write, execute
 V entry valid
 page frame page frame in physical memory

Mar 20, 2015

CSE312 Lecture 6

41

41

Page Faults

- If an invalid segment or page table entry is referenced, a page fault occurs
- The kernel looks at the offending address and decides what to do
- Perhaps it's a program bug — send an appropriate signal
- Perhaps the memory is paged out to disk — block the process, read in the page, fix up the page table, and retry

Mar 20, 2015

CSE312 Lecture 6

42

42

Other Control Bits

- Page referenced: used to decide which pages aren't in use right now, and hence can be paged out
- Modified: "dirty" pages have to be written to disk; unmodified pages may still be there from the last time they were paged out
- Memory protection: used to limit how some pages are used

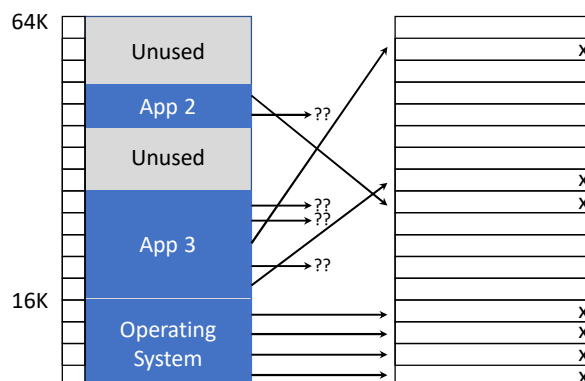
Mar 20, 2015

CSE312 Lecture 6

43

43

Unmapped Pages



Mar 20, 2015

CSE312 Lecture 6

44

44

Unmapped Pages

- Not all pages need to exist in main memory at the same time
- We can have a larger virtual memory than physical memory
- Again, locality of reference helps — most of an application isn't needed most of the time
- Deciding what pages to keep in memory is crucial to system performance

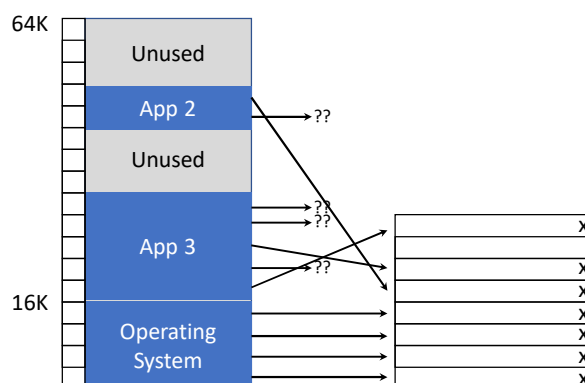
Mar 20, 2015

CSE312 Lecture 6

45

45

Less RAM Needed



Mar 20, 2015

CSE312 Lecture 6

46

46

Page Replacement Algorithms

- When a page fault occurs, some page that's currently in memory needs to be discarded to make room for a new page
- Picking the right page to discard isn't easy
- Many algorithms have been studied

Mar 20, 2015

CSE312 Lecture 6

47

47

Page Replacement Algorithms

- Optimal page replacement algorithm
- Not recently used page replacement
- First-In, First-Out page replacement
- Second chance page replacement
- Clock page replacement
- Least recently used page replacement
- Working set page replacement
- WSClock page replacement

Mar 20, 2015

CSE312 Lecture 6

48

48

Idealized Algorithm

- Ideally, we want to discard the page that will be needed last
- Which is that?
- Run the program again, find out which it is; that will tell you which to replace this time
- Not good...

Mar 20, 2015

CSE312 Lecture 6

49

49

Constraints

- Must be efficient — many paging decisions take place
- Must approximate the right answer
- Must be implementable on real hardware
- Usually, must work well on multitasking systems

Mar 20, 2015

CSE312 Lecture 6

50

50

Tools

- The OS has a few tools available to it
- The referenced bit — this page has been used recently
- The modified bit — discarding this page will be more expensive
- Clock interrupts
- Page fault interrupts
- Advice from the application

Mar 20, 2015

CSE312 Lecture 6

51

51

Page Fault Interrupts

- Enter the kernel
- Synchronous to the running process
- Instruction is retryable or restartable (what if the instruction's operand crosses a page boundary?)
- Fault information indicates the offending address as well as the restart point

Mar 20, 2015

CSE312 Lecture 6

52

52

Sometimes R and M are Missing

- Some hardware platforms omit the M and R bits
- They can be simulated in software
- To imitate M, periodically mark the page read-only; if there's a protection fault, the page should be considered "dirty"
- To imitate R, mark the page invalid; if there's a page fault, it should be considered "referenced"

Mar 20, 2015

CSE312 Lecture 6

53

53

Not Recently Used (NRU)

- At process start time, reset all R and M bits
- On clock interrupts, clear R bits
- Classify pages by M and R:

	R	M
Class 0:	0	0
Class 1:	0	1
Class 2:	1	0
Class 3:	1	1

- On page fault, discard a random page from the lowest class

Mar 20, 2015

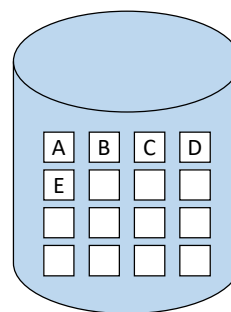
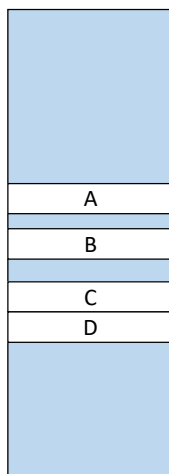
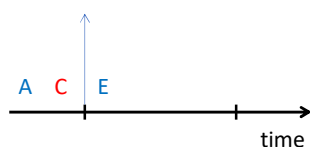
CSE312 Lecture 6

54

54

NRU Example

...	R	M	...	...
A	0	0		
B	0	0		
C	0	1		
D	0	0		



Mar 20, 2015

CSE312 Lecture 6

55

55

Resetting R and M

- Why do we reset R on clock interrupts?
 - We want to know if a page has been used recently
- Why not reset M?
 - M cannot be reset until the page has been written out to disk; an old copy will not be sufficient

Mar 20, 2015

CSE312 Lecture 6

56

56

Properties of NRU

- Bias towards discarding unmodified pages
- But — better to discard a modified page that has not been used recently than one that is in use
- Simple algorithm; may give adequate performance on some systems
- Primarily useful for teaching

Mar 20, 2015

CSE312 Lecture 6

57

57

Reclaiming Modified Pages

- Must first schedule a disk write operation
- When it is complete, the page frame can be reused
- Do we keep state binding a particular inbound page to that page frame?
- Do we make the other process wait for two disk operations before we let it run?
- Reclaiming a page is not cheap!

Mar 20, 2015

CSE312 Lecture 6

58

58

What's Interesting about NRU?

- It has the essential properties of any page replacement algorithm
- It looks for a (relatively) idle page
- It handles modified pages, but is biased against using them
- It is reasonably efficient

Spring 2026

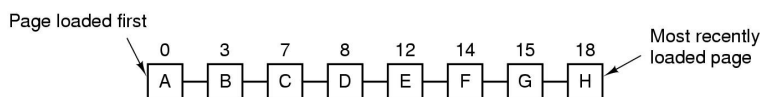
CSE312 Lecture 6

59

59

First In, First Out (FIFO)

- Forget about R and M
- When a page frame is needed, discard the oldest page
- Of course, the oldest page may still be busy, so it will come right back in
- FIFO is rarely used in this form



Mar 20, 2015

CSE312 Lecture 6

60

60

The Belady Anomaly

- Sometimes, having more page frames hurts instead of helps
- An example of Belady's anomaly
 - Using three page frames, 9 page faults occur.
 - Increasing to four page frames causes 10 page faults to occur.
 - Page faults are in **red**
- Very counter-intuitive
- Using FIFO page replacement

Page Requests	3	2	1	0	3	2	4	3	2	1	0	4
Newest	3	2	1	0	3	2	4	4	4	1	0	0
		3	2	1	0	3	2	2	2	4	1	1
Oldest			3	2	1	0	3	3	3	2	4	4

Page Requests	3	2	1	0	3	2	4	3	2	1	0	4
Newest	3	2	1	0	0	0	4	3	2	1	0	4
		3	2	1	1	1	0	4	3	2	1	0
			3	2	2	2	1	0	4	3	2	1
Oldest				3	3	3	2	1	0	4	3	2

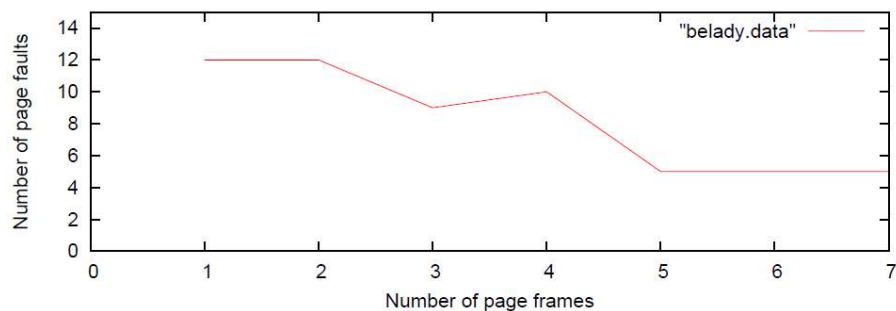
Mar 20, 2015

CSE312 Lecture 6

61

61

The Belady Anomaly



Mar 20, 2015

CSE312 Lecture 6

62

62

Second Chance FIFO

- Similar to pure FIFO, but the R bit is checked
- If R is set on an old page, clear the R bit but move it to the just-loaded end of the queue
- Approximates somehow LRU behavior
 - Note the problem: it doesn't know if a page has been used recently or not
 - It only approximates that if page faults are frequent

Mar 20, 2015

CSE312 Lecture 6

63

63

Second Chance FIFO

- Loaded pages are ordered in a FIFO list: "oldest" page first in list
- When oldest page selected for replacement, the reference bit is inspected
 - If value is 0: replace page
 - If value is 1: page gets a "second chance":
 - Clear reference bit, set to 0
 - Move page to tail of FIFO list
 - Check next-oldest page
- A page given a second chance will move to the tail of the FIFO queue and becomes the youngest page
- Worst case:
 - All reference bits == 1, degenerates into pure FIFO

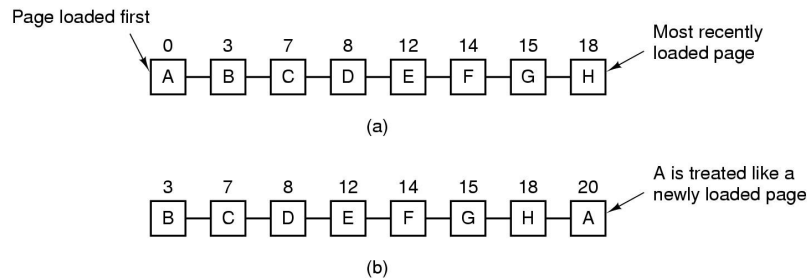
Mar 20, 2015

CSE312 Lecture 6

64

64

Second Chance Algorithm



Operation of second chance.

- (a) Pages sorted in FIFO order
- (b) Page list if a page fault occurs at time 20 and A has its R bit set. The numbers above the pages are their load times.

Mar 20, 2015

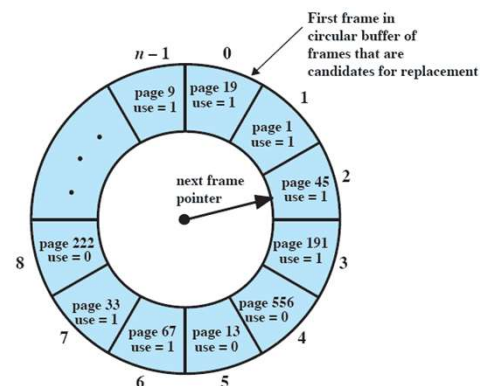
CSE312 Lecture 6

65

65

The Clock Algorithm

- FIFO can be implemented with a circular linked list; the list head is changed, rather than moving the page table entry → the clock algorithm
- The set of frames is considered as laid out like a circular buffer
 - contains a FIFO list of pages
 - can be circulated in a round-robin fashion
- Each page has a reference bit
 - when a page is first referenced (a page fault that leads to its load), the use bit of the frame is set to 1
- Each subsequent reference to this page again sets this bit to 1



Mar 20, 2015

CSE312 Lecture 6

66

66

The Clock Algorithm

- Initially, all frames in circular buffer are empty
- Frame pointer: is regarded the “clock hand”, is called the “next frame pointer”
 - After a page is loaded, it is moved forward to the next frame
 - When the circular buffer is filled with pages, the next frame pointer has made one full circle
 - it will be back at the first page loaded
 - points to the “oldest” page 1

Mar 20, 2015

CSE312 Lecture 6

67

67

Clock Policy

- Finding a page to replace
 - The “next frame pointer” advances from frame to frame in this circular “frame buffer”
 - As long as the “next frame pointer” encounters a page in a the frame with the reference bit set to 1, it is set it to 0 and moves on – it gives the page a second chance
 - The first page encountered with reference bit == 0 is replaced
 - Page is replaced, pointer is advanced to next page

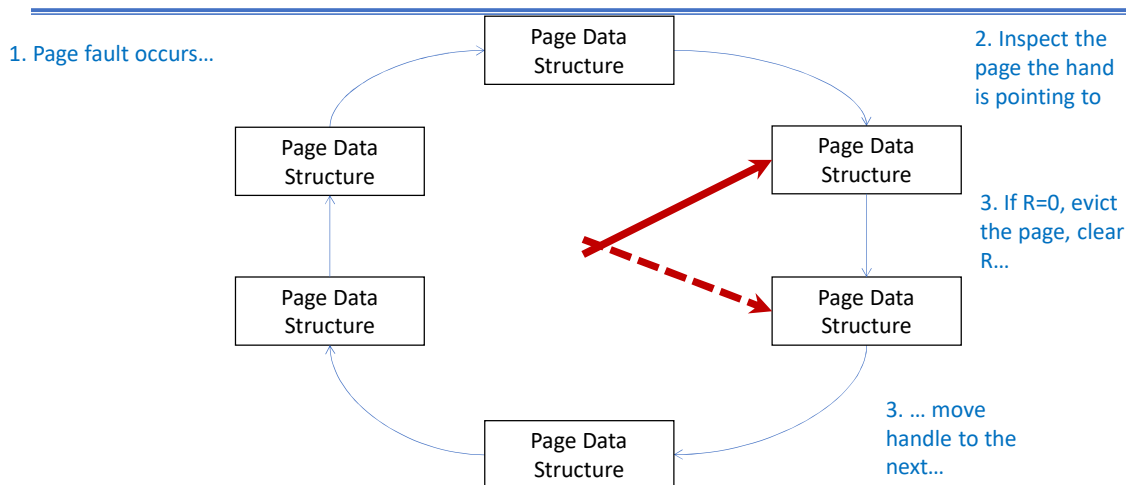
Mar 20, 2015

CSE312 Lecture 6

68

68

The Clock Algorithm



Mar 20, 2015

CSE312 Lecture 6

69

69

Least Recently Used (LRU)

- Assumption: a page that hasn't been used recently is unlikely to be used soon
- But — how can this be implemented?
- More precisely, what data structure do we use to track this?
- LRU is generally a linked list; do we point to it with a hash table indexed by page address?

Mar 20, 2015

CSE312 Lecture 6

70

70

Hardware-Assisted LRU

- Have a 64-bit instruction counter
- On each memory reference, store the counter in a per-page frame field
- On a page fault, scan the page table for the lowest value
- Note: this is $O(n)$ in the number of page frames

Mar 20, 2015

CSE312 Lecture 6

71

71

A Cheaper Hardware Assist

- For n pages, create a $n \times n$ bit matrix initialized to 0
- When referencing page k , set row k to 1 and set column k to 0
- The lowest binary value is the least-recently used page
- Again, $O(n)$ in the number of page frames

Mar 20, 2015

CSE312 Lecture 6

72

72

LRU Page Replacement Algorithm

	Page			
	0	1	2	3
0	0	1	1	1
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0

(a)

Set row k to all 1 and column k to 0...

LRU using a matrix when pages are referenced in the order 0, 1, 2, 3, 2, 1, 0, 3, 2, 3

Mar 20, 2015

CSE312 Lecture 6

73

73

Software-Simulated LRU

- Have an array of counters, one per page
- At each clock tick, add the value of R to the counter
- Implements NFU — Not Frequently Used
- Problem: never forgets

Mar 20, 2015

CSE312 Lecture 6

74

74

Simulating LRU in Software

	R bits for pages 0-5, clock tick 0	R bits for pages 0-5, clock tick 1	R bits for pages 0-5, clock tick 2	R bits for pages 0-5, clock tick 3	R bits for pages 0-5, clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 0	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00100000	10010000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000
	(a)	(b)	(c)	(d)	(e)

The aging algorithm simulates LRU in software. Shown are six pages for five clock ticks. The five clock ticks are represented by (a) to (e).

Mar 20, 2015

CSE312 Lecture 6

75

75

NFU With Aging

- Shift each counter right (i.e., divide by 2) before each addition
- Add R to the high-order bit
- Recent references have more weight
- Pages referenced this clock tick and previous clock ticks are more important than those referenced only this time
- Note: still $O(n)$, every clock tick

Mar 20, 2015

CSE312 Lecture 6

76

76

Working Set

Mar 20, 2015

CSE312 Lecture 6

77

77

Frame Allocation to Processes

- Each process is allocated free memory
 - Each process has a Free-frame list
 - Demand-paging will allocate frames from this list due to page faults
 - When the free-frame list is exhausted, a page replacement algorithm will choose a page to be replaced
- With multiple processes: how many free frames should be allocated to each process?

Mar 20, 2015

CSE312 Lecture 6

78

78

Minimum Number of Frames

- Performance depends on a minimum number of frames allocated
 - If number of free frames decreases, page faults increase
 - Increase of page faults slows process execution
 - I/O operations
 - Instruction that leads to a page fault has to be restarted (effectively executing instructions twice)
- We need enough frames to hold all the pages in memory that are referenced by any single instruction – principle of locality
- We have to manage the size of the resident set

Mar 20, 2015

CSE312 Lecture 6

79

79

Resident Set

- Is the number of pages currently loaded into physical memory
- Is restricted by the number of frames a process may maximally use

Mar 20, 2015

CSE312 Lecture 6

80

80

Resident Set Size

- **Fixed-allocation**
 - A process has a fixed pool of frames in main memory
 - If this pool is exhausted, a page fault will cause a page replacement
- **Variable-allocation**
 - Allow the pool of frames to vary in size over the lifetime of a process

Mar 20, 2015

CSE312 Lecture 6

81

81

Replacement Scope

- **We can classify page replacement in terms of allocation behaviour**
 - **Global page replacement**
 - A process selects a frame for replacement from a global list of frames (frames of all processes)
 - A page replacement will interfere with another process' memory allocation
 - **Local page replacement**
 - A process can only select a frame for replacement from its own local list of frames

Mar 20, 2015

CSE312 Lecture 6

82

82

Replacement Scope

- **Disadvantages**
 - **Global page replacement**
 - Performance of a process depends external circumstances, as other processes may replace its pages
 - Waste of performance
 - **Local page replacement**
 - Process is restricted to its own allocation of physical memory
 - Even if memory is not used by other processes, it is not available globally
 - Waste of resources
- Global replacement generally results in better system throughput and is the more commonly used method

Mar 20, 2015

CSE312 Lecture 6

83

83

Resident Set Management

	Local Replacement	Global Replacement
Fixed Allocation	• Number of frames allocated to a process is fixed. • Page to be replaced is chosen from among the frames allocated to that process.	• Not possible.
Variable Allocation	• The number of frames allocated to a process may be changed from time to time to maintain the working set of the process. • Page to be replaced is chosen from among the frames allocated to that process.	• Page to be replaced is chosen from all available frames in main memory; this causes the size of the resident set of processes to vary.

Mar 20, 2015

CSE312 Lecture 6

84

84

Thrashing

- Thrashing is a situation where a process is repeatedly page faulting
 - The process does not have enough frames to support the set of pages needed for fully executing an instruction
 - It may have to replace a page that itself or another process may need again immediately
- A process is thrashing if it is spending more time on paging than execution

Mar 20, 2015

CSE312 Lecture 6

85

85

Thrashing

- Limit the effects of thrashing by using a local replacement algorithm
 - Processes can only be served from their allocated pool of free memory frames
 - Process cannot “steal” frames from other processes
- Problem
 - When process starts thrashing, it will occupy the paging device queue most of the time
 - This increases the time for other processes to be served by the paging device

Mar 20, 2015

CSE312 Lecture 6

86

86

Prevent Thrashing

- If we want to prevent thrashing, we must provide a process with as many frames as it “needs”
- How many frames are needed?
- Principle of Locality
 - A “locality” is a set of pages that are **actively** used together during program execution
 - A program is generally composed of several different localities that may overlap
 - As a process executes, it moves from locality to locality

Mar 20, 2015

CSE312 Lecture 6

87

87

Prevent Thrashing

- Allocate frames for all pages of a locality
 - If all pages of the current locality are in memory, a process can execute without page faults
 - Page faults may occur only after it changes to another locality
- If we do not allocate enough frames to accommodate all pages of the current locality, a process will thrash
 - Process experiences frequent page faults as it cannot keep all those pages in memory it actively uses

Mar 20, 2015

CSE312 Lecture 6

88

88

Working Set

Mar 20, 2015

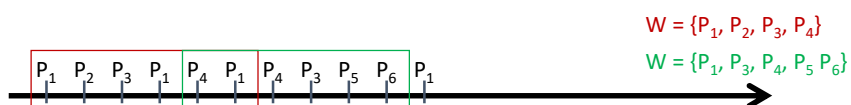
CSE312 Lecture 6

89

89

Working Set

- A working set is an approximation of a program's locality
- Uses the most recent page references from a program to determine which pages are most important
 - Parameter k : the most recent k page references
- Defines a “working set window”
 - The set of pages that are referenced within the recent k page references
 - A page in active use is part of the working set window
 - If a page is not used for k time units, it will be dropped from the working set window after these k time units



Mar 20, 2015

CSE312 Lecture 6

90

90

Working Set

- How many processes can be started?
 - Operating system will start processes and allocate frames as needed by their working sets
 - Working sets can change in size and need for frames
 - If the sum of all needed frames exceeds the actually available number of frames, one of the processes is selected for suspension
 - Its pages are swapped out and its frames reallocated to other processes
 - The suspended process can be restarted later, when enough frames become available again

Mar 20, 2015

CSE312 Lecture 6

91

91

The Working Set

- At any time, a program is only using a small fraction of its pages (locality of reference)
- The set of pages in use at the moment is the working set
- At time t , the working set $w(k, t)$ is the k most recently-referenced pages
- Note that $w(k, t)$ is monotonically increasing as a function of k and it asymptotically approaches the total program size
- A program needs to have its working set in memory
- If there isn't enough memory to hold the entire working set, the program will thrash

Mar 20, 2015

CSE312 Lecture 6

92

92

Page-Fault Frequency

- Use page-fault frequency to control thrashing
 - Thrashing has a high page fault rate
- Idea
 - If page-fault rate is high, then process needs more frames
 - If page-fault rate is low, then process has too many frames
- We can establish upper and lower bounds on the desired page-fault rate, controls frame allocation
 - If actual page-fault rate exceed upper limit, process receives an extra frame
 - If actual page-fault rate falls below lower bound, process loses one frame
- If there are no more frames to allocate, a process exceeding the upper bound, will be suspended and its frames reallocated to other processes

Mar 20, 2015

CSE312 Lecture 6

93

93

The Working Set

- At any time, a program is only using a small fraction of its pages (locality of reference)
- The set of pages in use at the moment is the working set
- At time t , the working set $w(k, t)$ is the k most recently-referenced pages
- Note that $w(k, t)$ is monotonically increasing as a function of k and it asymptotically approaches the total program size
- A program needs to have its working set in memory
- If there isn't enough memory to hold the entire working set, the program will thrash

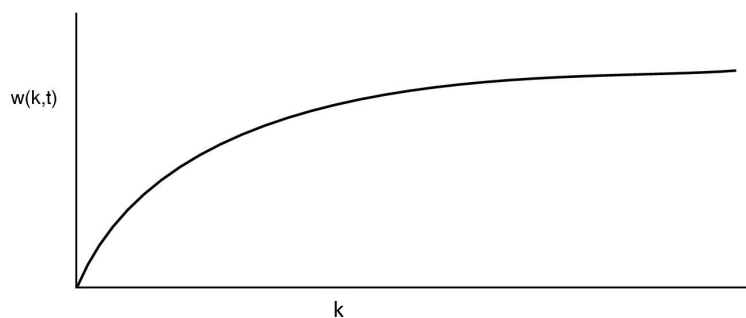
Mar 20, 2015

CSE312 Lecture 6

94

94

Working Set Page Replacement



The working set is the set of pages used by the k most recent memory references. The function $w(k, t)$ is the size of the working set at time t .

Mar 20, 2015

CSE312 Lecture 6

95

95

Using the Working Set

- Before running a process, make sure that the working set is in memory
- Because k is asymptotic, the exact choice of k isn't critical, as long as it's large enough
- The trick is to determine the working set

Mar 20, 2015

CSE312 Lecture 6

96

96

Approximating the Working Set

- Ideally, we would track the last k memory references
- Instead, we track pages referenced during the last τ seconds
- For each page, keep a clock field
- At each tick, update the clock field if R is set
- Pages not referenced during the last τ seconds may be discarded
- Algorithm is still $O(n)$. But we can do better.

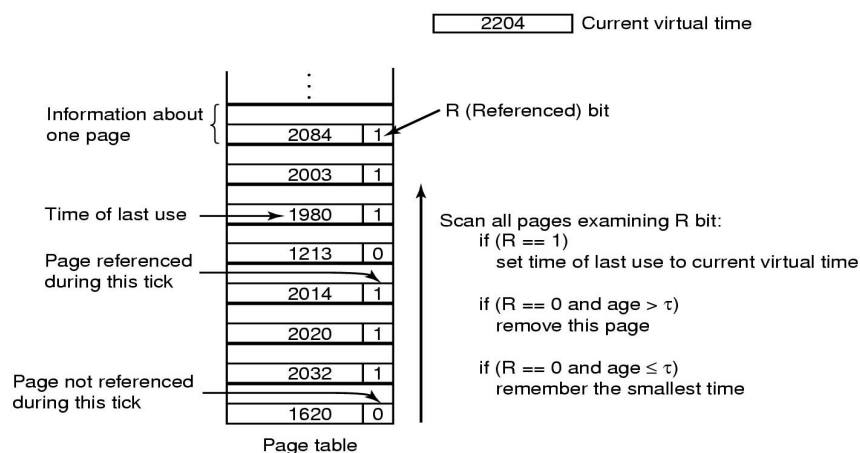
Mar 20, 2015

CSE312 Lecture 6

97

97

Working Set Page Replacement



Mar 20, 2015

CSE312 Lecture 6

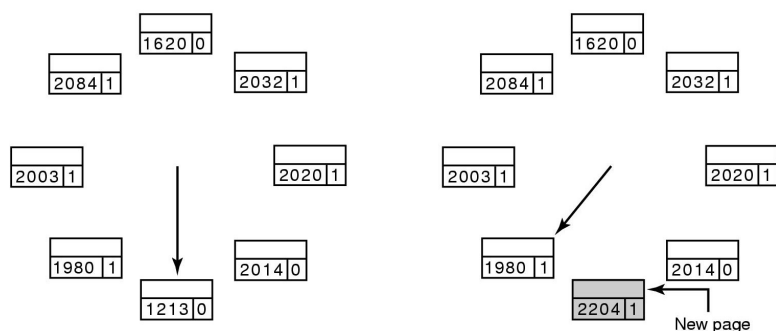
98

98

99

100

The WSClock Page Replacement Algorithm



When $R = 0$

Mar 20, 2015

CSE312 Lecture 6

101

101

Dealing with Modified Pages

- If $M = 0$, the page frame exists on disk and can be reused immediately
 - We initialize M to 1 if it's never been written
- If $M = 1$, schedule it to be written to disk and keep scanning — you might find a clean page
- Limit the number of writes for any pass to some value n

Mar 20, 2015

CSE312 Lecture 6

102

102

What Happens if the Clock Hand Circles?

- If some writes have been scheduled, keep looking; a write will complete eventually
- If no writes have been scheduled, all pages are in the working set; pick a random clean page to reuse
- But — the current process no longer has its working set in memory. Do we schedule it?
- In general, we have to be very cautious here; we don't want the total system to thrash

Mar 20, 2015

CSE312 Lecture 6

103

103

System Issues

Mar 20, 2015

CSE312 Lecture 6

104

104

The Scheduler and Paging

- As noted, the WSClock algorithm can force the scheduler to leave the CPU idle
- This will generally improve total throughput
- Also note that the pager generally needs a process context — it may want to wait for disk I/O to complete, for example

Mar 20, 2015

CSE312 Lecture 6

105

105

Anticipatory Paging

- The system can only reclaim a page immediately if it already exists on disk; otherwise, it has to wait for a disk operation to finish
- When the disk subsystem is relatively idle, find and write out dirty, unused pages
- The M bit can then be reset, making them immediately available for reuse if necessary

Mar 20, 2015

CSE312 Lecture 6

106

106

Prepaging

- When a process blocks for an extended period, remember its working set
- Its pages will tend to be reclaimed, since they're not being used
- When the process wakes up, restore its working set to memory before resuming execution
- Avoid the overhead of too many page faults

Mar 20, 2015

CSE312 Lecture 6

107

107

The Paging Disk

- You want the paging disk to be as fast as possible
- Sometimes, dedicate a separate disk to paging; avoid contention from file I/O
- Old systems often used drums or head-per-track disks, to minimize or avoid rotational delay or seek time

Mar 20, 2015

CSE312 Lecture 6

108

108

Sharing Page Tables

- If two processes are running the same executable and code pages are read-only, the two can share the same page table
- Do they share the same working set data structure? What if a page is in one process' working set but not the other's?
- Extra data structures are needed to track such shared pages; reference counters are mandatory

Mar 20, 2015

CSE312 Lecture 6

109

109

Page-Faulting in Executables

- Suppose that files with executables are formatted like page areas?
- That is, each in-memory page of the program begins on a disk block boundary
- Metadata and data areas are in different blocks
- No need to read in a program before executing it; just set up the paging data structures to point to the actual file
- This is one reason an executing file can't be opened for output

Mar 20, 2015

CSE312 Lecture 6

110

110

Implementing Copy on Write

- Processes might be using identical copies of same information – sharing for optimization
- What happens when one processes updates a portion of the shared data
- Copy on Write: The kernel
 - Determines when a change to the shared information is attempted
 - Creates a private copy of the information to make the changes
 - An consolidates...

Mar 20, 2015

CSE312 Lecture 6

111

111

Implementing Copy on Write

- Lets see how we can implement it
- The code pages are shared
- The two processes share data pages but not the page table for the data area
- The shared pages are marked read-only in both processes
- When a protection fault occurs, the page is copied to an empty page frame (where does it come from?), and both page table entries are updated

Mar 20, 2015

CSE312 Lecture 6

112

112

Locking Pages

- Real-time systems can't afford page faults, at least on parts of the program
- Solution: let the application lock some pages into memory
- Also used for some I/O
- If too many pages are locked, it can affect overall system performance
- Locking is thus restricted, either to root or to a certain quota

Mar 20, 2015

CSE312 Lecture 6

113

113

Summary of Page Replacement Algorithms

Algorithm	Comment
Optimal	Not implementable, but useful as a benchmark
NRU (Not Recently Used)	Very crude approximation of LRU
FIFO (First-In, First-Out)	Might throw out important pages
Second chance	Big improvement over FIFO
Clock	Realistic
LRU (Least Recently Used)	Excellent, but difficult to implement exactly
NFU (Not Frequently Used)	Fairly crude approximation to LRU
Aging	Efficient algorithm that approximates LRU well
Working set	Somewhat expensive to implement
WSClock	Good efficient algorithm

- NRU is never actually used
- Plain FIFO works poorly; second chance FIFO works reasonably well
- NFU with aging is a good choice
- WSClock is efficient and gives good results

Mar 20, 2015

CSE312 Lecture 6

114

114

Design Issues

Mar 20, 2015

CSE312 Lecture 6

115

115

Local versus Global Allocation

- When process A has a page fault, where does the new page frame come from?
- More precisely, is one of A's pages reclaimed, or can a page frame be taken from another process?
- If another process, do we bias the selection in any fashion?
- If page replacement affects only the current process, we have a local policy; if we look at all processes, we have a global allocation policy

Mar 20, 2015

CSE312 Lecture 6

116

116

Local versus Global Allocation Policies

	Age		
A0	10	A0	
A1	7	A1	
A2	5	A2	
A3	4	A3	
A4	6	A4	
A5	3	A6	
B0	9	B0	
B1	4	B1	
B2	6	B2	
B3	2	B3	
B4	5	B4	
B5	6	B5	
B6	12	B6	
C1	3	C1	
C2	5	C2	
C3	6	C3	

(a) Original configuration (b) Local page replacement (c) Global page replacement

Mar 20, 2015

CSE312 Lecture 6

117

117

Choosing

- Global policies tend to work better
- If you use a local policy and the working set grows, you can get thrashing
- Similarly, if the working set shrinks, you waste memory
- With a global policy, though, you need to decide how much memory to allocate to each process

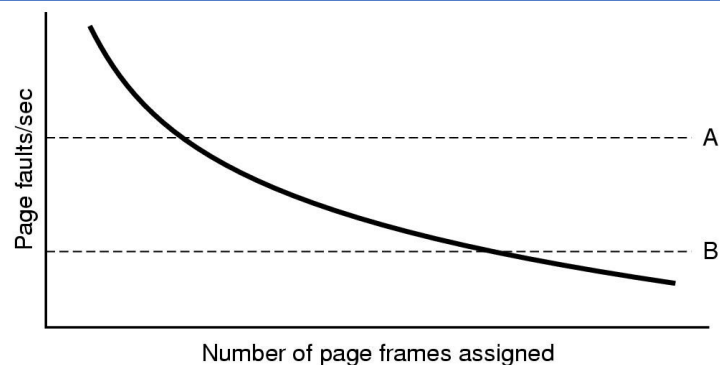
Mar 20, 2015

CSE312 Lecture 6

118

118

Local versus Global Allocation Policies



Page fault rate as a function of the number of page frames assigned

Belady?

Mar 20, 2015

CSE312 Lecture 6

119

119

Memory Allocation

- Fixed allocation – the same amount of space for all processes ($a = \frac{m}{p}$)
— is too simplistic
- Better idea — allocate each process some memory in proportion to its size:

$$a_i = m \cdot \frac{m_i}{\sum_{i=1}^p m_i}$$

- Do we want to use size or working set?
- What about process priority?

$$a_i = m \cdot \frac{f_{prio}(m_i)}{\sum_{i=1}^p m_i}$$

Mar 20, 2015

CSE312 Lecture 6

120

120

Memory Requirements Change

- Processes grow and shrink
- Working sets grow and shrink
- Allocations must be changed over time
- Monitor the page fault frequency (PFF) for each process
- A process with a high PFF gets a larger allocation; a process with a small PFF gets a smaller allocation

Mar 20, 2015

CSE312 Lecture 6

121

121

Measuring PFF

- Count the number of page faults per second
- Accumulate this as a moving average, of the type we've seen several times before
- For many algorithms, including LRU, PFF goes down as memory allocation increases

Mar 20, 2015

CSE312 Lecture 6

122

122

Algorithms versus Allocation

- Algorithms such as LRU and FIFO work with either local or global allocation policies
- Working set and WSclock are local-only
- There's no such thing as a working set for the entire system
- Must rely on allocation policy for global effects

Mar 20, 2015

CSE312 Lecture 6

123

123

Swapping

- As mentioned, the paging system has to interact with the scheduler
- Thrashing can be detected when the PFF rate of some processes has gone up, but none have gone down
- Must swap some processes to disk: write out all (or most) of their pages and reclaim their page frames

Mar 20, 2015

CSE312 Lecture 6

124

124

Controlling Swapping

- Which processes should get swapped out?
- Do we look at priority? Size? History?
- Once processes are swapped out, when do they come back in?
- Need a two-level scheduler, one for ordinary CPU access and one for swapping out and in
- For this second scheduler, what are we optimizing for? CPU utilization? Throughput?

Mar 20, 2015

CSE312 Lecture 6

125

125

Page Size

- With large pages, we waste memory: on average, half of the last page isn't used
- With small pages, we use a lot of memory for page tables
- Call the average process size s and the page size p . Assume that each page table entry (and associated data structures) takes e bytes
- The overhead $o = \frac{s}{p}e + \frac{p}{2}$
- To optimize for memory use, differentiate and set to 0:

$$\frac{d_o}{d_p} = -\frac{se}{p^2} + \frac{1}{2} = 0$$

- Best size: $p = \sqrt{2se}$

Mar 20, 2015

CSE312 Lecture 6

126

126

Simulating Larger Pages

- Possible to treat several smaller pages as one larger page
- Still need separate hardware page table entries, but can reduce overhead elsewhere
- Big gain: fewer page faults
- Other gains: auxiliary data structures

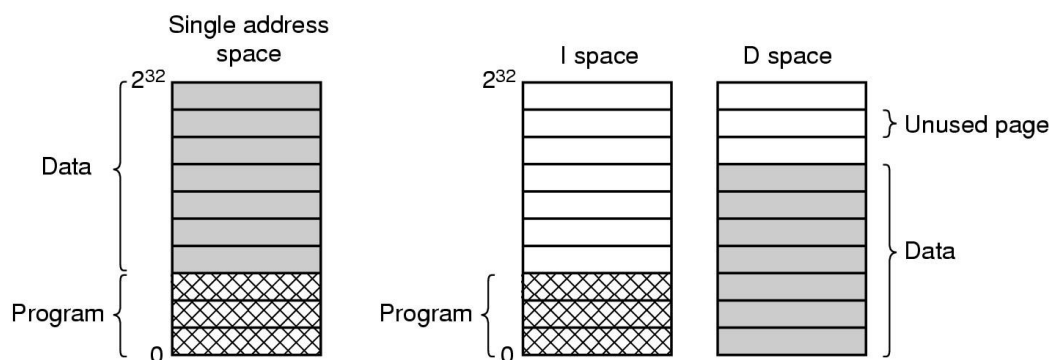
Mar 20, 2015

CSE312 Lecture 6

127

127

Separate Instruction and Data Spaces



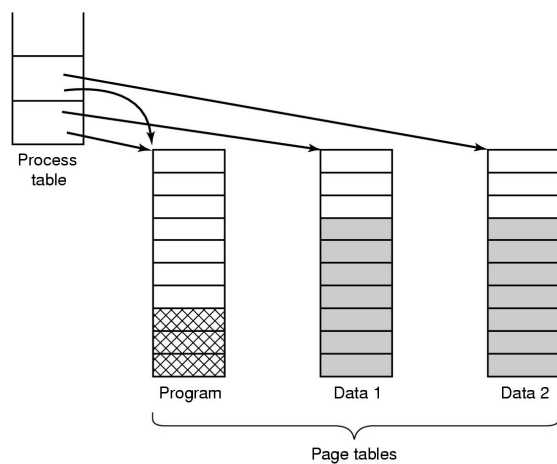
Mar 20, 2015

CSE312 Lecture 6

128

128

Shared Pages



Two processes sharing the same program sharing its page table

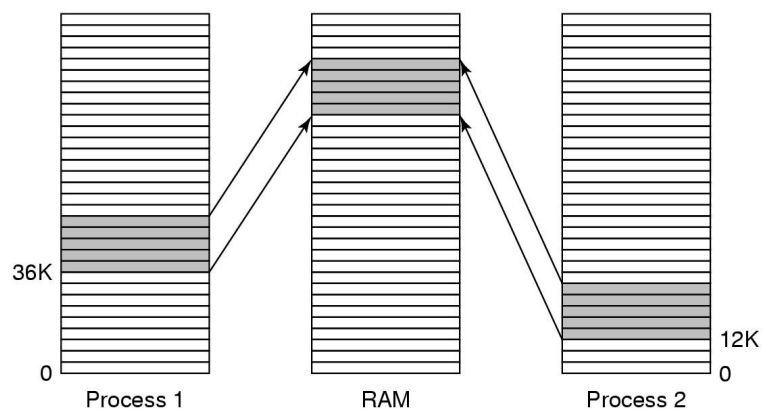
Mar 20, 2015

CSE312 Lecture 6

129

129

Shared Libraries



A shared library being used by two processes

Mar 20, 2015

CSE312 Lecture 6

130

130

Page Sharing and Remapping

- Context switch overhead can be reduced by page-sharing
- Example: shared memory in Unix (shmat(), shmget(), etc.) share memory between processes
- Caution: processes must use appropriate locks
- Comm pages allow processes to read (some) kernel data
- Example: getpid() can be a simple subroutine

Mar 20, 2015

CSE312 Lecture 6

131

131

Memory-Mapped I/O

- Instead of doing I/O, processes map a file onto a memory area, i.e., mmap()
- Easy random access
- Let the page algorithm handle the I/O
- On Multics, there was no disk I/O; all files were simply areas of memory
- Disadvantage: file size was limited by address space (actually, by segment size)

Mar 20, 2015

CSE312 Lecture 6

132

132

Page-Mapped I/O

- Suppose a user I/O buffer is page-sized and page-aligned
- Make sure that kernel disk buffers are page-sized and page aligned, too
- When the user process does a read(), change the page table so that the disk buffer is mapped to user space and the user's buffer becomes part of kernel memory
- No overhead for copying!
- Harder to do for write() — does the user process still want access to its data?
- Can sometimes “lend” pages, but mark them read-only

Mar 20, 2015

CSE312 Lecture 6

133

133

Don't Copy!

- Copying bulk data is very expensive
- Limited by memory bandwidth; could use a lot of cache space
- It's worth considerable effort to avoid handling data extra times

Mar 20, 2015

CSE312 Lecture 6

134

134

Allocating Swap Space

- Where does swap space come from?
- Some systems allocate swap space as soon as the application is given main memory
- In other words, all of the memory of every process has a reserved spot on disk
- Other systems allocate space as needed
- What if they run out?

Mar 20, 2015

CSE312 Lecture 6

135

135

Storage for Disk Mapping

- Where is the disk block address stored for a page?
- Some systems reuse the page table entry if the “valid” bit is off
- Works poorly if there’s a lot of swap space
- Doesn’t work if you keep the disk images of pages in case they’re not dirty when reclaimed

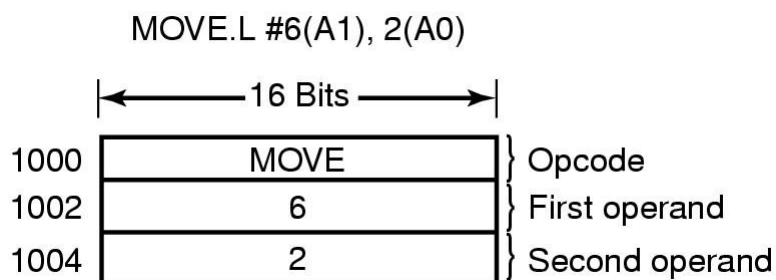
Mar 20, 2015

CSE312 Lecture 6

136

136

Instruction Backup



An instruction causing a page fault

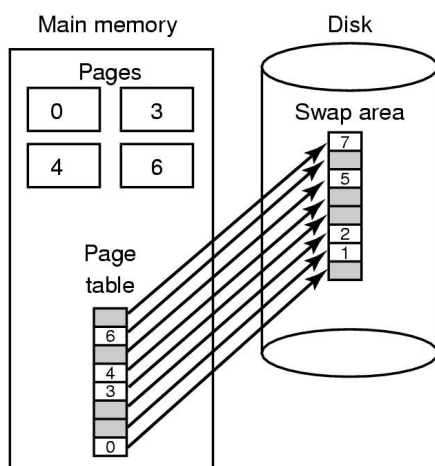
Mar 20, 2015

CSE312 Lecture 6

137

137

Backing Store



Paging to a static swap area

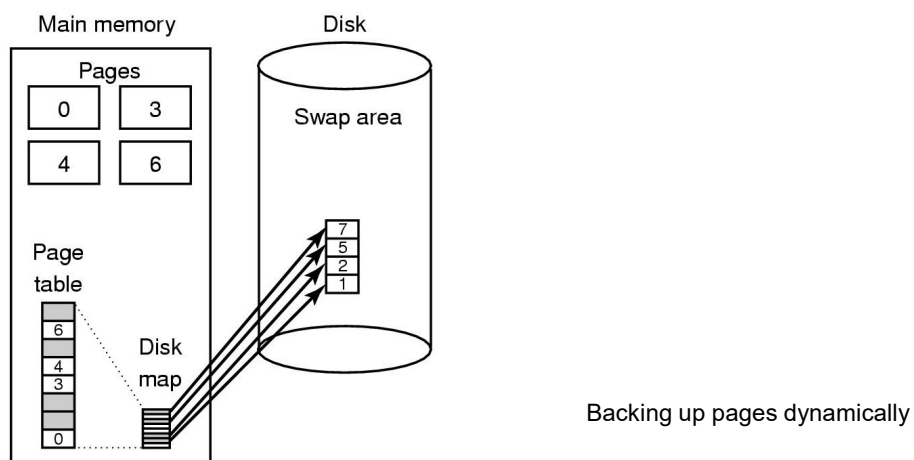
Mar 20, 2015

CSE312 Lecture 6

138

138

Backing Store



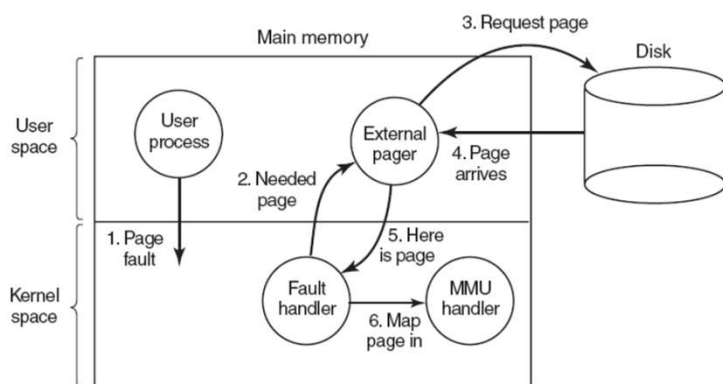
Mar 20, 2015

CSE312 Lecture 6

139

139

Separation of Policy and Mechanism



Page fault handling with an external pager

Mar 20, 2015

CSE312 Lecture 6

140

140

Segmentation

Mar 20, 2015

CSE312 Lecture 6

141

141

Logical Segmentation

- Earlier, we talked about segments for VM
- There's another type: user-controlled segments
- Segments introduce non-linearity into the address space
- There's no carry into the segment bits when doing address arithmetic

Mar 20, 2015

CSE312 Lecture 6

142

142

Why Use Segments?

- Code, data, and the stack are each separate segments
- Shared libraries can each occupy a separate segment
- That way, only the segment pointer needs to be separate; the same page table can be used for each process using the library
- In Multics, each file was mapped to a particular segment

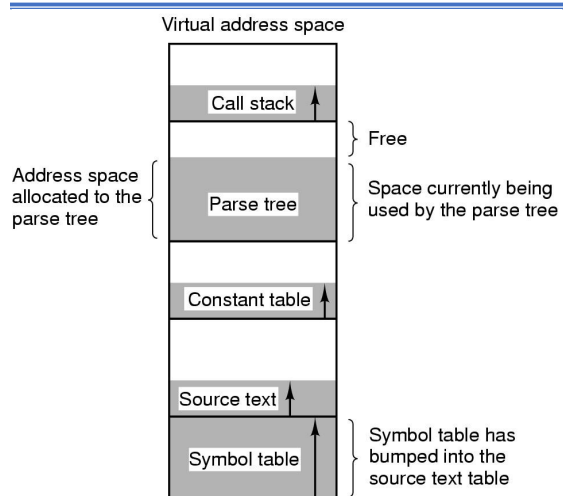
Mar 20, 2015

CSE312 Lecture 6

143

143

Segmentation



In a one-dimensional address space with growing tables, one table may bump into another

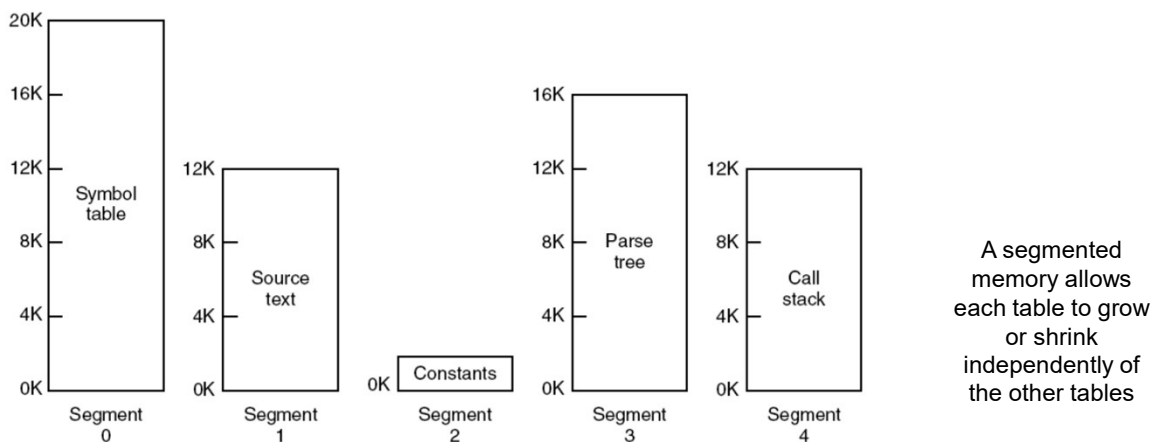
Mar 20, 2015

CSE312 Lecture 6

144

144

Segmentation



Mar 20, 2015

CSE312 Lecture 6

145

145

Protection and Segments

- Segments can have memory protection bits associated with them
- For the uses just described, this is more convenient and more natural than protecting each page independently

Mar 20, 2015

CSE312 Lecture 6

146

146

The Problems with Segments

- Maximum contiguous address space is limited by segment size
- For example, on the Intel 286, segments were limited to 64K; that meant that no array could be larger than 64K bytes
- Explicit segments are not often used today

Mar 20, 2015

CSE312 Lecture 6

147

147

Implementation of Pure Segmentation

Consideration	Paging	Segmentation
Need the programmer be aware that this technique is being used?	No	Yes
How many linear address spaces are there?	1	Many
Can the total address space exceed the size of physical memory?	Yes	Yes
Can procedures and data be distinguished and separately protected?	No	Yes
Can tables whose size fluctuates be accommodated easily?	No	Yes
Is sharing of procedures between users facilitated?	No	Yes
Why was this technique invented?	To get a large linear address space without having to buy more physical memory	To allow programs and data to be broken up into logically independent address spaces and to aid sharing and protection

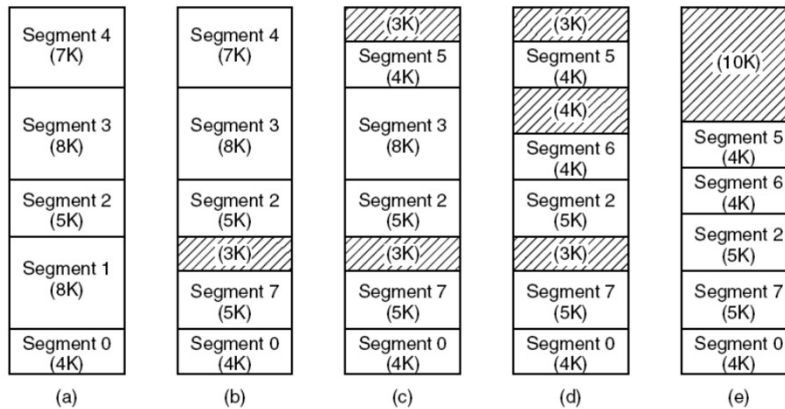
Mar 20, 2015

CSE312 Lecture 6

148

148

Segmentation with Paging: MULTICS



(a)-(d) Development of checkerboarding. (e) Removal of the checkerboarding by compaction.

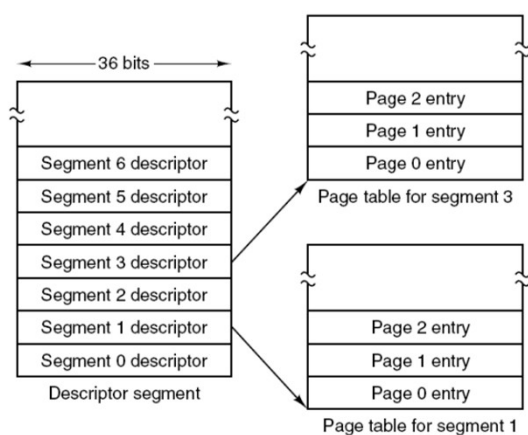
Mar 20, 2015

CSE312 Lecture 6

149

149

Segmentation with Paging: MULTICS



The MULTICS VM: The descriptor segment points to the page tables.

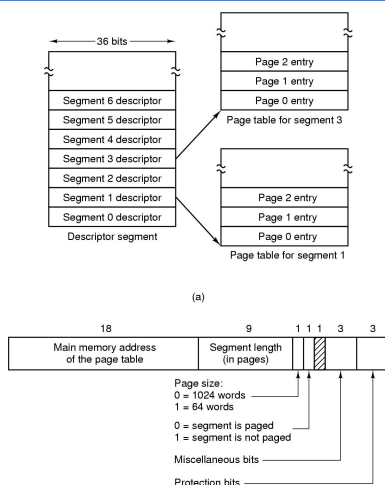
Mar 20, 2015

CSE312 Lecture 6

150

150

Segmentation with Paging: MULTICS



The MULTICS VM: A segment descriptor. The numbers are the field lengths

Mar 20, 2015

CSE312 Lecture 6

151

151

Segmentation with Paging: MULTICS

- When a memory reference occurs, the following algorithm is carried out:
 - The segment number used to find segment descriptor.
 - Check is made to see if the segment's page table is in memory.
 - If not, segment fault occurs.
 - If there is a protection violation, a fault (trap) occurs.
 - Page table entry for the requested virtual page examined.
 - If the page itself is not in memory, a page fault is triggered.
 - If it is in memory, the main memory address of the start of the page is extracted from the page table entry
 - The offset is added to the page origin to give the main memory address where the word is located.
 - The read or store finally takes place.

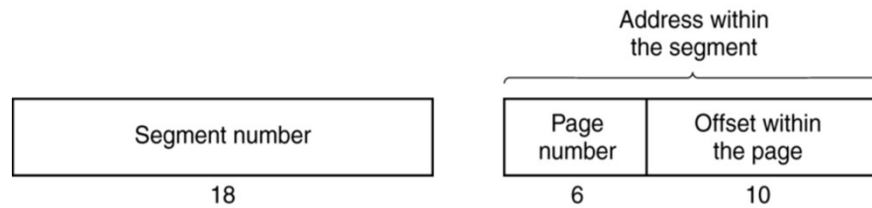
Mar 20, 2015

CSE312 Lecture 6

152

152

Segmentation with Paging: MULTICS



A 34-bit MULTICS virtual address

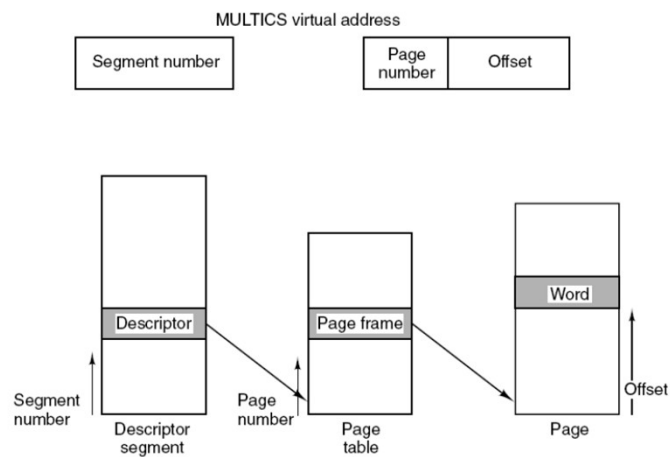
Mar 20, 2015

CSE312 Lecture 6

153

153

Segmentation with Paging: MULTICS



Conversion of a two-part MULTICS address into a main memory address

Mar 20, 2015

CSE312 Lecture 6

154

154

Segmentation with Paging: MULTICS

Comparison field		Page frame	Protection	Age	Is this entry used?
Segment number	Virtual page				
4	1	7	Read/write	13	1
6	0	2	Read only	10	1
12	3	1	Read/write	2	1
					0
2	1	0	Execute only	7	1
2	2	12	Execute only	9	1

A simplified version of the MULTICS TLB. The existence of two page sizes makes the actual TLB more complicated

Mar 20, 2015

CSE312 Lecture 6

155

155

Segments on the Pentium

- Six segment registers: code, data, four others
- 8K system and 8K user segments permitted
- A segment descriptor contains a base/limit pair and a pointer to memory
- That memory address may be virtual, in which case two levels of page table are used
- Three extra memory look-ups per memory reference!
- Good thing we have a TLB. . .

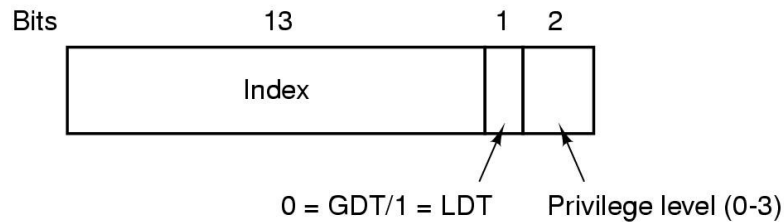
Mar 20, 2015

CSE312 Lecture 6

156

156

Segmentation with Paging: The Pentium



A Pentium selector

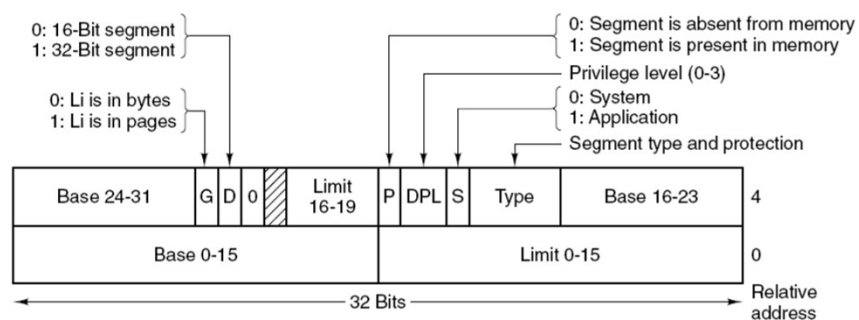
Mar 20, 2015

CSE312 Lecture 6

157

157

Segmentation with Paging: The Pentium



Pentium code segment descriptor. Data segments differ slightly.

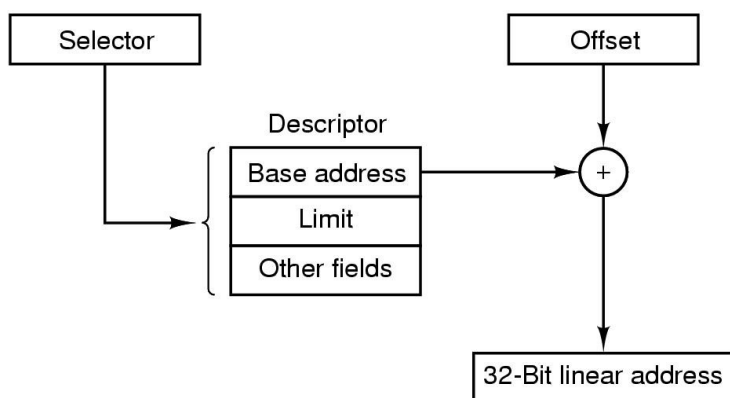
Mar 20, 2015

CSE312 Lecture 6

158

158

Segmentation with Paging: The Pentium



Conversion of a (selector, offset) pair to a linear address

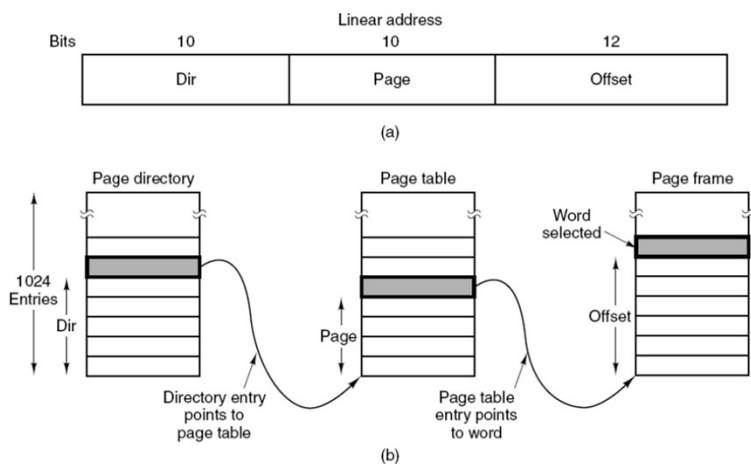
Mar 20, 2015

CSE312 Lecture 6

159

159

Segmentation with Paging: The Pentium



Mapping of a linear address onto a physical address

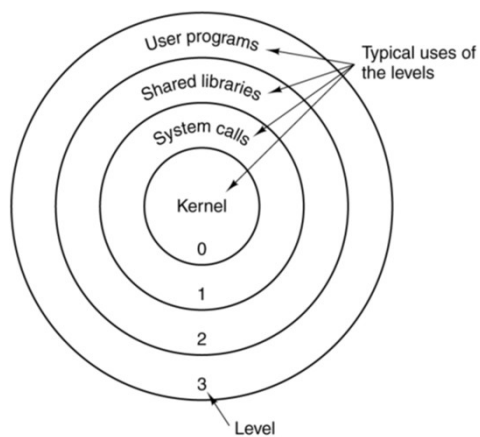
Mar 20, 2015

CSE312 Lecture 6

160

160

Segmentation with Paging: The Pentium



Protection on the Pentium

Mar 20, 2015

CSE312 Lecture 6

161

161

Memory Allocation

Mar 20, 2015

CSE312 Lecture 6

162

162

Types of Memory

- Kernel code — wired down (but some systems have used disk-resident system calls that are swapped in as needed)
- User code — paged in and out
- Page tables — must be dynamically allocated
- Stacks — also dynamically allocated
- Disk I/O buffers
- Network I/O buffers

Mar 20, 2015

CSE312 Lecture 6

163

163

Network I/O Buffers

- Allocation can be fixed
- If a user process writes too much, block
- If a remote process writes too much, use flow control to make it shut up
- If it doesn't listen, drop packets

Mar 20, 2015

CSE312 Lecture 6

164

164

Disk I/O Buffers

- How much memory should be allocated for disk I/O buffers?
- Simplest solution: some fixed percentage of memory
- Better solution: dynamically use memory for disk or for applications, as needed
- When system goes I/O-bound, leave fewer pages for applications
- When system is memory-bound, use fewer pages for disk I/O
- Sounds good, but getting the balance right is tricky

Mar 20, 2015

CSE312 Lecture 6

165

165

Kernel Memory Allocation

- Many kernel routines need to allocate memory dynamically
- Similar to malloc() for application programs
- These routines generally grab pages
- If there are no page frames free, the request can fail
- Often, there is a process context, and the process can block while waiting for a page
- Sometimes, the memory reclamation daemon is told to speed up

Mar 20, 2015

CSE312 Lecture 6

166

166

Managing Kernel Memory

- Kernel memory allocation is very similar to non-VM memory region allocation and malloc() allocation
- But — some systems will change the kernel's memory map to permit creation of large contiguous memory regions

Mar 20, 2015

CSE312 Lecture 6

167

167

Modeling Paging Systems

Mar 20, 2015

CSE312 Lecture 6

168

168

A Theory of Paging?

- Can we figure out a theory of paging?
- Can we predict performance?
- Can we explain — or prevent — things like Belady's Anomaly?

Mar 20, 2015

CSE312 Lecture 6

169

169

Reference Strings

- A process can be characterized by an ordered list of the pages it accesses
- This is called the reference string
- A paging system can be described by three things: the reference string of the process, the page replacement algorithm, and the number of page frames available

Mar 20, 2015

CSE312 Lecture 6

170

170

An Abstract Model

- The process we're modeling has n pages
- There are m page frames
- Assume an n -element array M that keeps track of memory
- $M[n - m : n - 1]$ represents in-memory pages
- $M[0 : n - m - 1]$ contains all other pages

Mar 20, 2015

CSE312 Lecture 6

171

171

Simulating the Process

- Take an entry from the reference string
- If the top half of M has room, put the page in it
- Otherwise, there's a page fault
- Apply the selected algorithm to move a page from the top of M to the bottom
- Move the new page to the top half
- Rearrange the top and bottom halves according to the algorithm

Mar 20, 2015

CSE312 Lecture 6

172

172

Simulating LRU

- Reference string is 0 2 1 3 5 4 6 3 7 4 7 3 3 5 5
- Four page frames available

0	2	1	3	5	4	6	3	7	4	7	3	3	5	5
0	2	1	3	5	4	6	3	7	4	7	3	3	5	5
	0	2	1	3	5	4	6	3	7	4	7	7	3	3
		0	2	1	3	5	4	6	3	3	4	4	7	7
			0	2	1	3	5	4	6	6	6	6	4	4
				0	2	1	3	5	4	6	6	6	6	4
					0	2	1	1	1	1	1	1	1	1
						0	0	2	2	2	2	2	2	2
							0	0	0	0	0	0	0	0
P	P	P	P	P	P	P		P					P	
∞	∞	∞	∞	∞	∞	∞	4	∞	4	2	3	1	5	1

Mar 20, 2015

CSE312 Lecture 6

173

173

Stack Algorithms

- If m varies over the possible page frames and r is an index into the reference strings, we may have

$$M(m, r) \subseteq M(m + 1, r)$$
- That is, for a given initial sequence of a reference string, those pages that are at the top of M will still be in the top of M if there is one more page frame
- Algorithms that satisfy this property are called stack algorithms
- Belady's Anomaly cannot occur with stack algorithms

Mar 20, 2015

CSE312 Lecture 6

174

174

Is LRU a Stack Algorithm?

- Whenever a page is pushed below the line in LRU, it goes to the top of the bottom section
- If we move the boundary down by one page frame, we therefore include the previously-displaced page
- That means that the stack property holds — LRU is indeed a stack algorithm
- FIFO is not

Mar 20, 2015

CSE312 Lecture 6

175

175

Distance Strings

- Assume a stack algorithm
- A distance string d is a set of page references where the value is “distance from the top of the stack”
- An unreferenced page isn’t on the stack and has distance ∞
- Distance strings are algorithm-dependent
- Small values are good; they indicate locality of reference
- You want most elements of d to be less than the number of page frames
- If d is mostly large numbers, you’re out of luck

Mar 20, 2015

CSE312 Lecture 6

176

176

Predicting Page Fault Rates

- Scan the distance string and see how many times each value occurs
- Let C_i be the number of times i is found; C_∞ exists, too
- For our example, $\langle C_1, C_2, \dots, C_7, C_\infty \rangle = \langle 4, 2, 1, 4, 2, 2, 1, 8 \rangle$
- If m is the number of page frames,

$$F_m = \sum_{k=m+1}^n C_k + C_\infty$$

- F_m is the number of page faults for that distance string and number of page frames

Mar 20, 2015

CSE312 Lecture 6

177

177

Origin of our Strings

- Where do reference and distance strings come from?
- As always, we can simulate them, but we're much better off getting real traces from real programs
- Note the implication: paging algorithm behavior can change if our workload changes

Mar 20, 2015

CSE312 Lecture 6

178

178

See you next time!